

Programmable Money — Product Requirements Document

A rules engine for coordinated money, built on Pollar (Stellar)

Pollar Hackathon — "Build on Pollar" · Prepared by Aka (Akachi Chukwuma) · Sep 16–18, 2026

1. The Product, in One Sentence

A programmable money app that turns financial routines into rules — starting with rotating Ajo circles and personal money allocations.

Two modes for the hackathon, one roadmap item beyond it:

- PERSONAL — "Set rules for where your money goes."
- AJO — "Coordinate recurring contributions and rotating payouts." (Hero / flagship demo)
- SHARED GOAL (roadmap only, not built this hackathon) — "Pool flexible contributions toward a shared goal, on a fixed date."

2. The Problem

Personal

People already know what they want their money to do. The problem is repeated manual execution — saving, paying bills, contributing to savings groups, moving money between accounts, every single time money comes in.

Ajo

Ajo groups already understand the model. The problem is that coordination and execution are manual: tracking contributions, remembering whose turn it is, calculating the pot, chasing people on WhatsApp.

3. Our Solution

A rules engine sitting on top of Pollar's wallet and transaction infrastructure.

Personal example

- Every 25th of the month: 20% → Savings, ₦30,000 → Ajo, ₦50,000 → Rent, remainder → Spending.

Ajo example

- 5 members, ₦10,000/week, rotating payout, predetermined order — the rules engine determines when a payout is eligible; the demo triggers execution manually.

4. What Pollar Does vs. What We Build

Verified directly against docs.pollar.xyz — critical for the dev team, since nobody should spend hours recreating something Pollar already provides, or assume it provides something it doesn't.

Pollar provides (SDK)	We build (our IP)
Login + wallet creation per user (<code>login({ provider })</code>)	Rules engine (Personal splits, Ajo cycles)
Sending — <code>runTx('payment', {...})</code> , one call per transfer	Contribution tracking & payout-order logic

Pollar provides (SDK)	We build (our IP)
Transaction history — <code>fetchTxHistory({ limit, offset })</code>	Circle / cycle state machine
Balances — <code>refreshWalletBalance()</code>	Validation (overspend, % totals, payout gating)
Earn (Blend / DeFindex yield)	Business-model layer (see §13)
Distribution Rules modal — CLAIM only, not push (see note below)	Multi-destination split execution (loop of <code>runTx</code> calls)

Important correction: Pollar's Distribution Rules feature (`DistributionRulesModal / listDistributionRules / claimDistributionRule`) is a CLAIM flow, not a push flow. You configure claimable rules in the Dashboard and users pull from a distribution wallet — there is no app-initiated "send to 5 people by percentage" primitive. This means our Personal-mode splitting and Ajo payouts must be built entirely as our own logic: one `runTx('payment', ...)` call per destination/recipient. Current understanding: Pollar's Distribution Rules APIs are claim-based. Our MVP should not depend on them for push-based multi-recipient execution. Re-test against the installed SDK version before implementation.

Starting point — don't build from scratch

Clone github.com/pollar-xyz/demo-nextjs as the project skeleton. It already has PollarProvider setup, login, send, and transaction history wired correctly — saves Hour 0–3 of boilerplate. Live reference: demo-nextjs.pollar.xyz (testnet).

5. User Journey — Personal

Login → Dashboard → Personal → Create Rule → Choose date/frequency → Add allocations → Preview → Validate → Activate → Rule saved → Execute manually (hackathon) → Transactions → Transaction history

Worked example — 300 USDC available balance

Destination	Rule	Amount
Savings	20%	₦60,000
Rent	Fixed ₦50,000	₦50,000
Ajo	Fixed ₦30,000	₦30,000
Spending	Remainder	₦160,000

The system previews this allocation before the user activates it.

6. User Journey — Ajo (Hero Demo)

Dashboard → Ajo → Create Circle → Set contribution → Set frequency → Add members → Set payout order → Circle created → Members contribute → Pot updates → All contributions received → Payout eligible → Recipient paid → Next cycle begins

Worked example — 5 members × 10 USDC

- Pot per cycle: ₦50,000
- Cycle 1 → Ada receives ₦50,000
- Cycle 2 → Chika receives ₦50,000
- ...continues through payout order

7. Screen Inventory

Not all of these are separate pages — several are states within one screen. Screens marked (MVP) are what we actually build this hackathon.

Shared

- Landing (MVP)
- Authentication — via Pollar's built-in login (MVP)
- Dashboard (MVP)
- Wallet (MVP — via Pollar WalletButton)
- Transactions (MVP — via Pollar TxHistoryModal)

Ajo

- Ajo overview (MVP)
- Create circle (MVP)
- Circle details / contribution status (MVP)
- Payout status + Execute Payout action (MVP)
- Cycle history (MVP, can be a tab within circle details)

Personal (stretch — build only if Ajo is fully working first)

- Personal overview
- Create rule / allocation builder
- Rule preview + activate

Shared Goal (roadmap — mockup only, not functional)

- One static screen: goal name, target, contributors, progress bar — no working logic behind it

8. MVP Definition

MUST WORK

- Pollar: auth, wallet, balance, one real send on testnet, transaction history
- Ajo: create circle, add members, track contributions, payout gate, execute payout, advance cycle

NICE TO HAVE

- Personal mode fully functional
- Countdown to next payout
- Notifications
- Polished empty/loading/error states

NOT BUILDING

- AI financial adviser · investment platform · lending · chat · social feed
- Full KYC flow (Pollar's KycModal is UI-preview only — do not depend on it)
- Mobile app · crypto exchange · cross-border LatAm demo (unless time allows, see §12)

9. Team Structure & Roles

Role	Owns	Notes
You (Product Lead)	Rules engine logic, Pollar integration proof-of-concept, final integration, demo/pitch	Test-run the Pollar SDK + rules engine alone before onboarding the team
UI/UX	Wireframes → hi-fi screens for MVP screens, empty/loading/error/success states	Design against the data model in §10, not from memory
Dev A — Pollar/Infra	Project setup, @pollar/react provider, wallet, send, tx history	Prove one real testnet transaction end-to-end first
Dev B — Product Logic	Ajo circle/cycle state, contribution tracking, payout gating, calling the rules engine	Rules engine functions should be pure — no direct Pollar calls

You + whichever dev is free own final integration — where Dev A's Pollar layer and Dev B's product logic meet is the highest-risk seam.

10. Data Model

TypeScript, not Python — Pollar's SDK is JS/TS-native (@pollar/core, @pollar/react). The rules engine logic below is simple enough to live entirely in TS alongside the frontend.

Entity	Fields
User	id, name, walletId (Pollar-managed)
PersonalRule	id, userId, frequency, executionDate, status, allocations[]
Allocation	destination, type: 'percentage' 'fixed', value
AjoCircle	id, name, contributionAmount, frequency, members[], payoutOrder[], currentCycle, status
Contribution	circleId, memberId, amount, cycle, status, pollarTransactionId
Payout	circleId, recipientId, cycle, amount, status, pollarTransactionId

11. The Rules Engine (Core IP)

Pure functions — no ML, no Python needed. They take state in and return a plan out; a separate thin executor layer loops over that plan calling Pollar's runTx('payment', ...) once per destination/recipient (confirmed: Pollar has no native multi-recipient split — see §4). This separation lets the math be unit-tested with zero Pollar dependency, and lets Dev A and Dev B build in parallel.

```
function executePersonalRule(income, allocations) { ...calculates amounts per
destination, validates total does not exceed income, returns { destinations,
remainder } } function checkAjoPayout(circle) { ...checks all members paid this
cycle; if not, returns locked; if yes, returns { recipient, amount } from
payoutOrder } // executor layer — the only part that touches Pollar async function
executePlan(plan, runTx) {   for (const { destination, amount, asset } of
plan.destinations) {       const result = await runTx('payment', { destination,
amount, asset });         if (result.status === 'error') { /* surface result.details,
do not continue silently */ }     } }
```

Validation the engine must enforce

- No negative remainder (allocations cannot exceed income)
- Percentage allocations must not sum past 100%

- Ajo payout only fires when: every member has contributed for the current cycle AND the current cycle has not already been paid — this is the single highest-risk piece of logic; read it yourself before demo day

Pollar-specific execution notes

- No scheduling primitive exists — a scheduled date is our own trigger (demo: manual button; production: cron/backend calling runTx)
- Each Ajo member should log in individually (or be simulated via 5 pre-created test accounts) so contributions are wallet-attributed by sender, not by parsing labels — resolves the attribution question from earlier in planning A contribution is considered received only after a successful payment is recorded for the correct circle, cycle, member, amount, and asset type.
- runTx does not throw on failure — always check `result.status === 'error'` and read `result.details/resultCode`; do not assume success

12. Hackathon Demo Script

Everyone on the team should be able to repeat this — one story, not five unrelated features.

Opening line

"We all know what we want our money to do. The problem is we keep manually doing it."

Demo 1 — Ajo (hero)

- Create circle → show 5 members × 10 USDC = 50 USDC pot
- Show contributions coming in, pot updating live
- Show payout locked until all 5 have paid
- Execute payout → real Stellar testnet transaction → show in transaction history

Demo 2 — Personal (if built)

- 300 USDC available balance → 20% Savings, ₦50k Rent, ₦30k Ajo, remainder Spending
- Preview → Activate → real transaction

Closing line

"Same rules engine. Two very different financial behaviours — and it extends to shared goals like a wedding fund or a family project, on the same engine, next." (Show the static Shared Goal mockup here, if built.)

13. Business Model Notes (for Q&A, not built this hackathon)

Ajo's cultural trust model means a visible per-transaction fee undermines the product. Monetization ideas, in order of fit:

- Yield/partner economics: Subject to custody, regulatory, product, and Pollar terms, eligible idle funds could potentially generate yield, with disclosed economics supporting the platform (future hypothesis, not MVP), and directly showcases an underused part of the SDK
- Spread on the fiat↔stablecoin on/off-ramp leg, not on the Ajo pot itself
- Freemium tiers (free for small circles; paid for scale, multiple circles, advanced reporting)
- B2B: cooperatives, churches, employers running many circles pay a flat SaaS fee

14. Hour-by-Hour Checkpoints

Checkpoint	Deliverable	If missed
Hour 0–3	You: clone demo-nextjs, get testnet key from dashboard.pollar.xyz, confirm login + one real send + tx history, working solo	Do not onboard the team until this works
Hour 3	Team onboarded with this PRD	—

Checkpoint	Deliverable	If missed
Hour 3–8	Ajo circle creation, contribution tracking, payout gate	Cut Personal mode entirely
Hour 8–12	Ajo payout execution + UI polish	Cut Shared Goal mockup
Hour 12+	Personal mode (if time), Shared Goal mockup (if time), demo rehearsal	Rehearse Ajo-only demo

Submission deadline: September 18, 2026, 1:00 PM UTC.

A member contribution is 'verified' only if: (1) successful transaction, (2) sender = member wallet, (3) amount $\geq$ required contribution, (4) asset = correct type, (5) circle = correct circle, (6) cycle = current cycle

Payment Adapter Pattern (Architecture for resilience)

To isolate Pollar SDK integration from core business logic and enable emergency fallback:

```
interface PaymentAdapter {
  sendPayment(params: {destination, amount, asset}): Promise<{hash, status}>
}
```

Real Adapter: PollarPaymentAdapter $\rightarrow$ Pollar $\rightarrow$ Stellar $\rightarrow$ testnet

Demo Adapter: DemoPaymentAdapter $\rightarrow$ mock deterministic hash, no blockchain

Benefit: If Pollar testnet breaks 2 hours before demo, switch the adapter configuration in one line. The Ajo logic, validation, and UI remain untouched.

MVP Safety Rule: The app ships with both adapters wired. REAL mode targets Pollar testnet (production demo). DEMO mode (labeled clearly in UI) is the fallback.

Definition of Done — Ajo MVP

- ☐ Circle creation UI works with 5 hardcoded members
- ☐ Each member's contribution status is independently tracked
- ☐ Pot total = sum of verified contributions (amount-matched)
- ☐ Payout remains LOCKED while any member is unpaid
- ☐ Correct recipient selected from payoutOrder[currentCycle]
- ☐ Duplicate payout prevented — button disabled during execution
- ☐ Payout executes via Pollar (real) or Demo adapter (fallback)
- ☐ Successful payout recorded with transaction hash
- ☐ Cycle advances exactly once after successful payout
- ☐ Failed payout does NOT advance the cycle
- ☐ Team can reproduce entire flow from clean start in <5 min

Contribution Verification Rule (Critical logic)

A member is marked 'paid' ONLY if ALL of these are true:

- Transaction executed and succeeded (status = 'SUCCESS')
- Sender wallet address matches member's registered wallet
- Amount $\geq$ required contribution amount
- Asset type matches expected asset (e.g., USDC native)
- Circle ID matches the contribution target
- Cycle number matches current cycle

IF all members are 'paid' AND current cycle has not been paid $\rightarrow$ payout eligible

ELSE $\rightarrow$ payout locked (no button execution)

Execution Modes: Real vs Demo Fallback

REAL mode: Pollar SDK $\rightarrow$ PollarPaymentAdapter $\rightarrow$ Stellar testnet $\rightarrow$ real transaction hash

DEMO mode: DemoPaymentAdapter $\rightarrow$ mock hash $\rightarrow$ UI shows '[DEMO]' badge, all logic works identically

MVP rule: Final demo uses REAL mode if Pollar is stable. Switch to DEMO mode only if Pollar authentication or testnet breaks. No code changes required — configuration switch only.

Transaction Safety (Partial failure & Idempotency)

For Personal rules with multiple destinations: each destination is a separate runTx() call. If one fails after others succeed, the plan state becomes PARTIAL_FAILURE and no remaining transfers are silently attempted. The UI shows which destinations succeeded and which failed.

For Ajo (MVP): only one payout transaction per cycle, so partial failure risk is minimal. But duplicate execution is prevented: once a payout executes, the cycle is marked 'paid' and the button is disabled.

Hour 0–2: Technical Spike with Decision Gate

You + Dev A work in parallel:

- Clone pollar-xyz/demo-nextjs
- Install @pollar/react, @pollar/core
- Set testnet publishable key in .env
- Wire PollarProvider at app root
- Confirm login works (Google or email OTP)
- Send ONE real testnet payment via sendPayment()
- Verify transaction appears in history

⚠ Decision at Hour 2:

If Pollar works: Continue to real integration (Hour 3–8). If Pollar blocks: Freeze SDK integration, build against DemoPaymentAdapter. Core Ajo logic remains unchanged either way.